

SOFTWARE ENGINEERING

Prof. Dr. Christoph Andriessens
christoph.andriessens@hs-weingarten.de



ANFORDERUNGEN UND ANFORDERUNGSMANAGEMENT

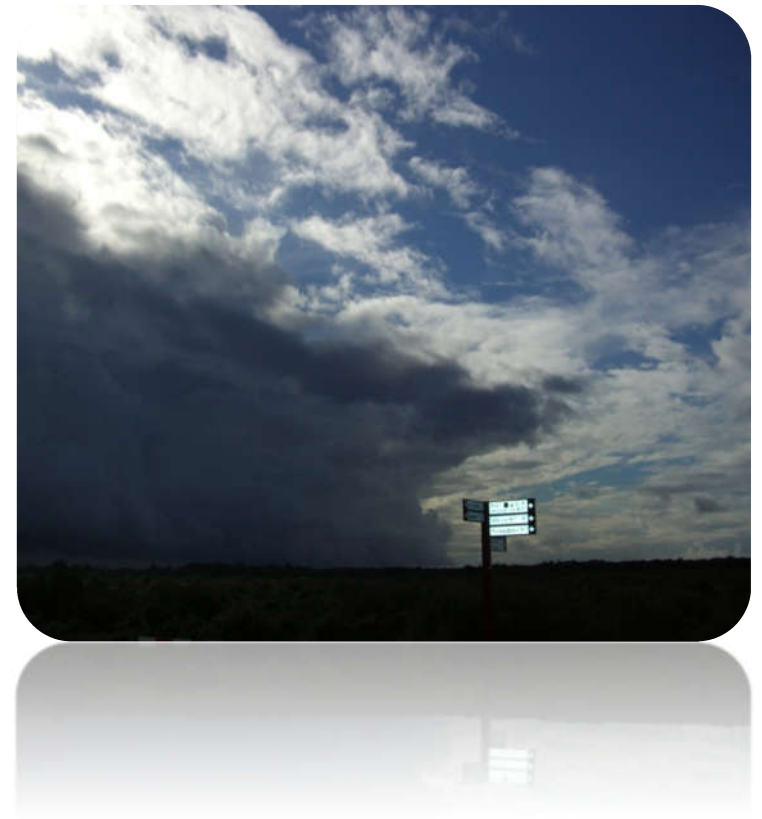
Oder: Die richtige Software bauen ... eine kleine Einführung

Themen in dieser Vorlesung



Lernziele

- Sie steigen in das Thema Anforderungen ein, lernen unterschiedliche Anforderungstypen kennen und können diese bewerten
- Sie lernen, anhand des Kano-Modells Anforderungen zu priorisieren
- Sie erhalten Hinweise zu typischen Fallstricken und Best Practices



Zu spät entdeckter Fehler in Anforderungen Mars Climate Orbiter

Home | Video | Themen | Forum | English | DER SPIEGEL | SPIEGEL TV | Abo | Shop | Schlagzeilen | Wetter | TV-Programm | mehr ▼

SPIEGEL ONLINE WISSENSCHAFT Login | Registrierung

Politik | Wirtschaft | Panorama | Sport | Kultur | Netzwelt | Wissenschaft | Gesundheit | einestages | Karriere | Uni | Reise | Auto | Stil

Nachrichten > Wissenschaft > Mensch > Mars Climate Orbiter: Absturz wegen Leichtsinnsfehler beim Rechnen

Mars Climate Orbiter: Absturz wegen Leichtsinnsfehler beim Rechnen

Nicht wegen einer technischen Panne, sondern weil die beteiligten Wissenschaftler in verschiedenen Maßeinheiten rechneten, ist die 125 Millionen teure Marssonde Climate Orbiter abgestürzt. Ein klassischer Schülerfehler führte bei der Übersetzung vom amerikanischen ins metrische Maßsystem zur peinlichsten Pleite der Nasa. Eine weitere Sonde ist vielleicht mit denselben Fehlberechnungen zum Mars unterwegs.

1 Freitag, 01.10.1999 - 12:24 Uhr

Drucken | Senden | Merken

Nutzungsrechte | Feedback

Mehr im Internet

Mars Climate Orbiter Official Website

Teilen | Empfehlen 0 | Twittern 0 | +1



Los Angeles - Ein schlichter Rechenfehler ist offenbar Schuld am Scheitern der jüngsten amerikanischen Mars-Mission. Nach Angaben der US-Raumfahrtbehörde Nasa gab es eine folgenschwere Verwechslung bei der Übertragung der Flugdaten vom amerikanischen ins international gebräuchlichere metrische System. Ein Fehler, der dazu geführt, dass der 125 Millionen teure

<http://www.spiegel.de/wissenschaft/mensch/mars-climate-orbiter-absturz-wegen-leichtsinnfehler-beim-rechnen-a-44777.html>

Kosten für Fehlerbehebung nach Entwicklungsphase

Normalisierte Kosten

Source	Phase Requirements Issue Found			
	Requirements	Design	Code	Test
[Boehm, 1981]	1	5	10	50
[Hoffman, 2001]	1	3	5	37
[Cigital, 2003]	1	3	7	51
[Rothman, 2000]		5	33	75
[Rothman, 2000] Case B			10	40
[Rothman, 2000] Case C			10	40
[Rothman, 2002]	1	20	45	250
[Pavlina, 2003]	1	10	100	1000
[McGibbon, 2003]		5		50
Mean	1	7.3	25.6	177
Median	1	5	10	50.5

Stecklein et. al.: *Error Cost Escalation Through the Project Life Cycle*. Nasa Technical Report, 2004.

Anforderungen

Definition

Eine Bedingung oder Fähigkeit, die eine Software erfüllen oder besitzen muss, um einen Vertrag, eine Norm oder ein anderes, formell bestimmtes Dokument zu erfüllen.

IEEE Standard Glossary of Software Engineering 610.12-1990

„Gute“ erhobene Anforderungen stellen fest:

1. Was ist *wirklich* das Problem?
2. Welche Art von Lösung wird die Kunden zufriedenstellen?

➤ Aber: Was ist gut? Das werden wir näher ansehen.

Anforderungen

Die zentrale Funktion von Anforderungen

Anforderungen brechen ein Projektziel herunter in messbare Einzelaufgaben. Sie sind Basis für:

- **Preisverhandlungen** zwischen Lieferant und Kunde
- **Vertragsschluß**
 - Klärung von Vertragsstreitigkeiten
- **Entwurf und Programmierung** – was soll umgesetzt werden?
- **Test** – was wurde richtig umgesetzt?
- **Projektsteuerung** – was ist schon umgesetzt?
 - Zeitplanung – was wird wann geliefert?
- **Abnahme** – der Kunde prüft, was er erhalten hat

Anforderungen

Darstellung in unterschiedlicher Detaillierung

- Darstellungen reichen von hoch-abstrakter Aussage über einen Service über Beschränkung für das System bis zu detaillierter mathematisch-funktionaler Spezifikation
- Diese Bandbreite ist unvermeidbar, da Anforderungen unterschiedlichen Funktionen erfüllen müssen, etwa:
 - Mal Basis für Angebotsabgaben bei Bewerbung um Projekt – müssen offen für verschiedene Lösungsvarianten (und damit Interpretation, aber nicht Ungenauigkeit!) sein
 - Mal Basis dann für den Vertrag – müssen detaillierter beschrieben sein



ANFORDERUNGEN UND ANFORDERUNGS- MANAGEMENT:

ARTEN VON ANFORDERUNGEN

Arten von Anforderungen



Arten von Anforderungen

Funktionale Anforderungen

Funktionale Anforderungen beschreiben, was ein Programm tun muss. Also Festlegungen zu

- konkreten Funktionen, die das System bereitstellen soll
- konkreten Reaktionen des Systems zu bestimmten Eingaben

Funktionale Anforderungen sind bei Umsetzung

- sofort sichtbar als neue Funktionalität
 - leichter testbar als Nicht-Funktionale Anforderungen
- Drücken sich in Quelltext (einzelnen Anweisungen) aus

Beispiel: Funktionale Anforderungen in einem medizinischem Informationssystem

- Ein Anwender soll in der Terminliste aller Kliniken (Fachabteilungen) suchen können
- Das System soll für jeden Tag für jede Fachabteilung eine Liste von Patienten erzeugen, die an diesem Tag zu einem Termin erwartet werden
- Jeder Mitarbeiter, der das System verwendet, soll eindeutig über seine 8-stellige Mitarbeiternummer identifiziert werden

Arten von Anforderungen

Nicht-funktionale Anforderungen

Nicht-funktionale Anforderungen beschreiben die Art und Weise, in der die funktionalen Anforderungen erreicht werden sollen oder müssen.

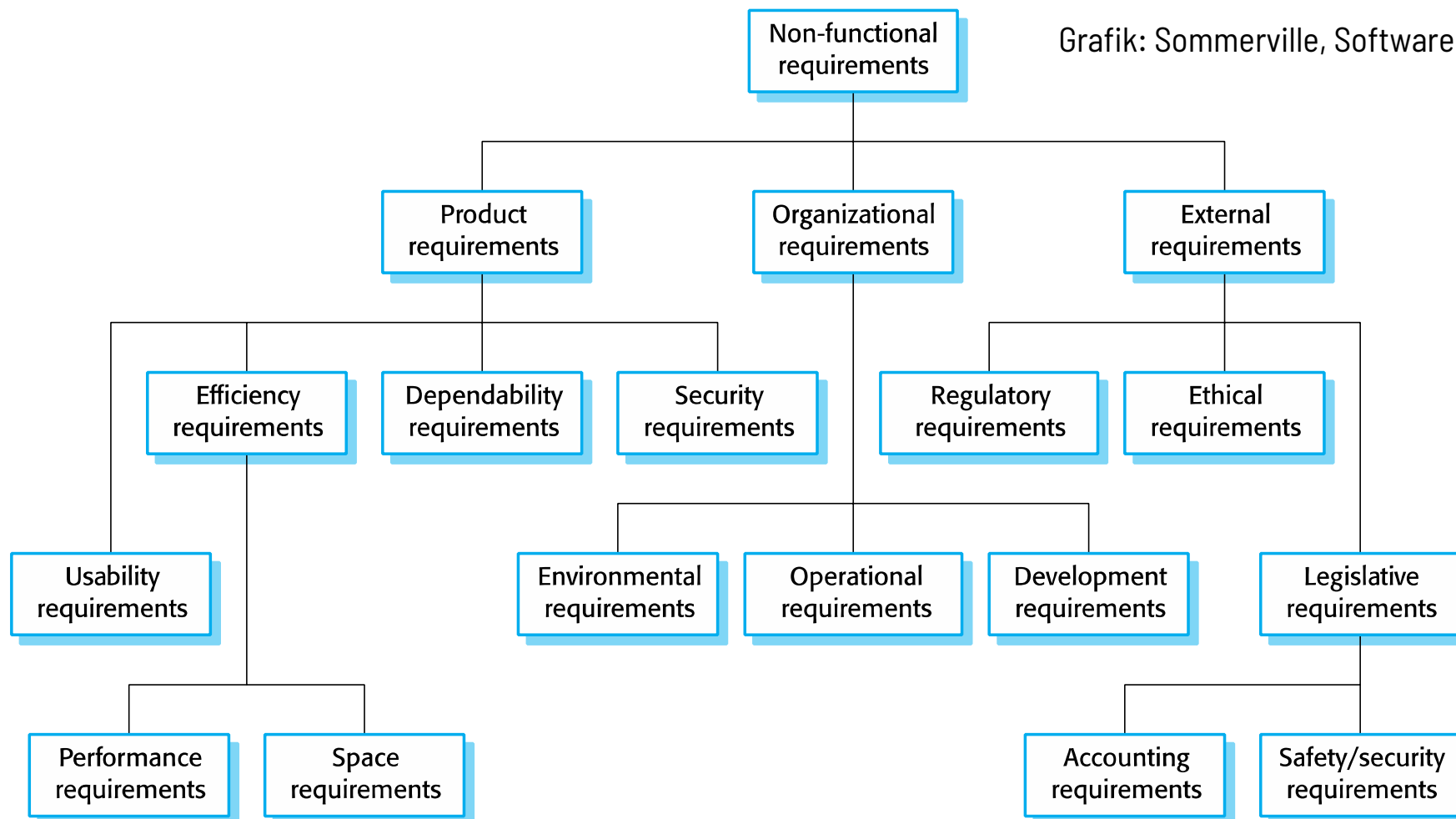
- Also: Vorgaben an die Umsetzung der funktionalen Anforderungen, gewissermaßen „Qualitätsanforderungen“
- Querschnittsanforderung: Performance, Stabilität / Ausfallsicherheit, Konfigurierbarkeit, Wartbarkeit, Sicherheit, ...

Nicht-Funktionale Anforderungen sind bei Umsetzung

- meistens verteilt über mehrere Stellen im System (viele Funktionen müssen performant, stabil, sicher, ... sein, damit das System es ist)
 - schwerer zu testen
- Drücken sich stark in Architektur (in der Struktur des Systems) aus

Typen nicht-funktionaler Anforderungen

Grafik: Sommerville, Software Engineering

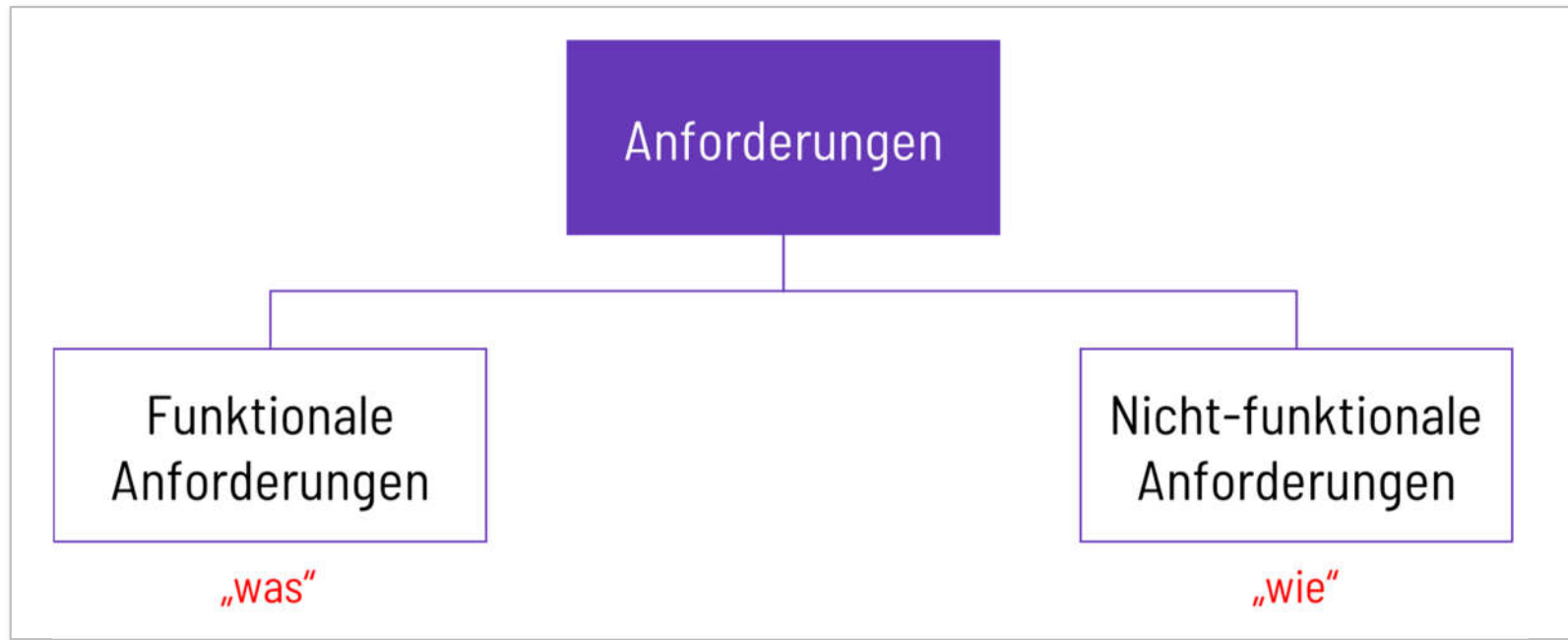


Beispiel: Nicht-funktionale Anforderungen in einem medizinischem Informationssystem

- **Produktanforderung**
 - Das Informationssystem soll für alle Fachbereiche während der normalen Arbeitszeiten verfügbar sein (Mo-Fr, 08:30-17:30). Die Downtime darf während normaler Arbeitszeiten pro Tag 5 Sekunden nicht überschreiten.
- **Organisatorische Anforderung**
 - Die Nutzer des Systems sollen sich mit ihrem Mitarbeiterausweis identifizieren.
- **Externe Anforderung**
 - Das System soll Datenschutz nach gesetzlichen Bestimmungen gewährleisten.

Arten von Anforderungen

Ist die Abgrenzung scharf?



Nicht alle Anforderungen sind klar „funktional“ oder „nicht-funktional“. Manche Anforderungen lassen sich auf beide Arten formulieren. Beispiel für zwei nicht-funktionale Anforderungsarten:

- Konfigurierbarkeit: „... diese Eigenschaft muss über ein Dialogfenster folgender Art ... folgendermaßen konfigurierbar sein: ...“
- Benutzerfreundlichkeit: Auch hier lassen sich Vorgaben funktional formulieren, etwa „das Programm zeigt eine Dialogbox mit 60 x 80 Pixeln an, und dann ...“

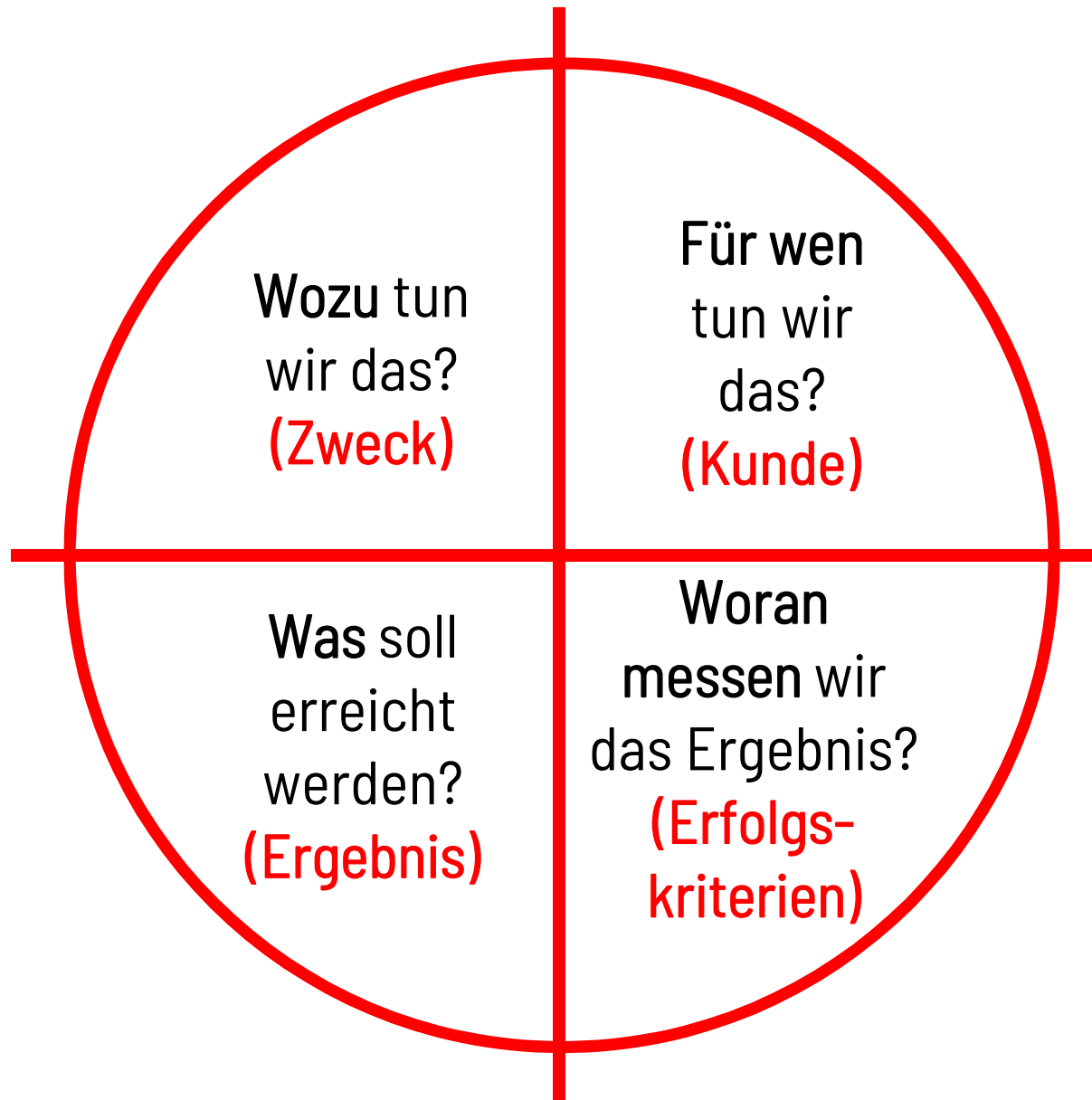
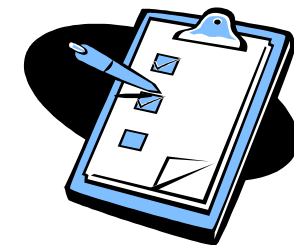


ANFORDERUNGEN UND ANFORDERUNGS- MANAGEMENT:

“GUTE” ANFORDERUNGEN,
ZIELE UND FAKTEN

Wie geht man vor?

Das große Ziel kennen



Zuerst Projektziele klären

– sind nicht immer klar!

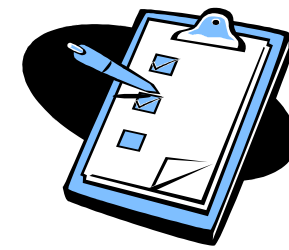
- Mit vier Fragen lässt sich jedes Ziel sauber klären
- „Zielkreuz“ zeigt: Sind nicht alle vier Fragen klar beantwortbar, ist das Ziel nicht ausreichend festgelegt!
- Auch Anforderungen sind Ziele ...

Wie geht man vor?

Ziele festlegen, nicht Weg



- Ziele der Funktionalität ermitteln:
 - Nicht: „Ich brauche ein Feld, um Wert X abzulegen!“ – sondern: was soll erreicht werden?
 - Wozu dient die Funktion? Wer benötigt das Ergebnis wozu?
 - Ziele ohne unnötige Einschränkungen formulieren
- Auf **Fakten** basieren lassen:
 - Existierende Berichte und Auswertungen, Listen zeigen lassen – ggf. hinterfragen
 - Nichts ungeklärt im Raum stehen lassen, Vermutungen gelten nicht!



„Gute“ Anforderungen

Was sind typische Probleme mit Anforderungen?

- **Ein typisches Problem:** Anforderungen können auf unterschiedliche Arten von Entwicklern und Anwendern verstanden werden
 - Nicht im Sinne unterschiedliche Lösungsvarianten
 - Sondern: Ungenau es Ziel oder ungenauer Weg
 - **Beispiel:** "suchen" bei der Klinik-Terminliste
 - Benutzerabsicht: Suchen eines Patientennamens in allen Terminen aller Fachabteilungen, gleichzeitig
 - Entwickler denkt: Suchen eines Patientennamens in einzelner Fachabteilung. Anwender wählt erst Fachabteilung, dann sucht er.
- **Weitere Probleme:** Eine Vielzahl von Anforderungen müssen im Umsetzungsstatus verfolgt werden, überschneiden oder widersprechen sich, sind nicht hinreichend abgestimmt, ...

„Gute“ Anforderungen

Qualitätskriterien: Das heißt „gut“



- **Vollständig:** Ohne Lücken
- **Atomar:** Eine, nicht zwei oder drei in einem Satz
- **Notwendig, gültig und aktuell**
- **Verfolgbar:** Was wird / wurde daraus? (IDs!)
- **technisch lösungsneutral:** Ziel formuliert ohne unnötige Einschränkungen
- **Verständlich:** Für Stakeholder *und* Entwickler

- **Realisierbar:** Mit annehmbaren Risiko
- **Konsistent:** Ohne Widersprüche
- **Eindeutig:** Nur auf eine Art und Weise verstehbar
- **Prüfbar:** Umsetzung nachweisbar durch Test oder Messung
- **Abgestimmt:** Abgenommen
- **Klassifizierbar** bezüglich juristischer Verbindlichkeit

Quelle: Rupp / SOPHISTen

Wie geht man vor?

Die Richtigen beteiligen



Stakeholder (dt. "Interessensgruppe, Anspruchsgruppe") bei Softwaresystemen oder Softwareprojekten:

Jede Person oder Organisation, die von dem System in irgendeiner Weise betroffen ist und so ein legitimes Interesse daran hat

Gehören zu folgenden Typen:

- Endanwender (darunter konkrete Gruppen wie Finanzbuchhaltung, Geschäftsführung, Lagerarbeiter, ...)
- System Manager
- System Owner (wer ist für System verantwortlich, wessen „Kind“ ist es?)
- Externe Stakeholder

Beispiel: Stakeholder in einem Medizinischem Informationssystem 1

- Patienten, deren Information im System gespeichert werden
- Ärzte, die für Diagnose und Behandlung von Patienten verantwortlich sind
- Krankenpfleger die pflegen, koordinieren und kleinere Behandlungen durchführen
- Personal am Empfang, das Termine koordiniert
- IT-Abteilung, die für Installierung und Betrieb des Systems zuständig ist

Beispiel: Stakeholder in einem Medizinischem Informationssystem 2

- **Datenschutzbeauftragter** der sicherstellen muss, dass die Patientendaten entsprechend gesetzlicher Bestimmungen gespeichert werden
- **Krankenhausverwaltung**, die Steuerungsinformationen aus dem System erhält
- **Medizinische Dokumentation** die sicherstellen muss, dass die Informationen im System ordnungsgemäß eingetragen sind und (lange) aufbewahrt werden können – auch abrechnungsrelevant

Wie geht man vor?

Den Kunden integrieren



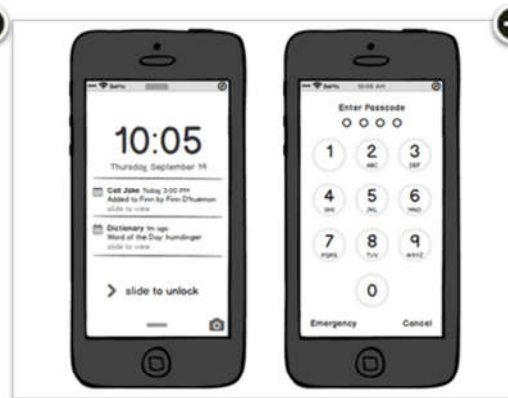
- Viel und frühes **Feedback**
 - Direktes Feedback über Änderungen / Entscheidungen an Kunden zurückgeben: Was sind die Konsequenzen?
 - Benutzeroberfläche mit Kunden zusammen entwerfen
- Geschäftsprozesse an Oberfläche **durchspielen**
- Alle Anforderungen **priorisieren** –
aber 90% Prio A ist sinnlos
- **Gemeinsam festlegen**, wo sich das System am wahrscheinlichsten verändern könnte

Kunden integrieren

Wireframing: Frühes Feedback



Desktop Apps



Mobile Apps



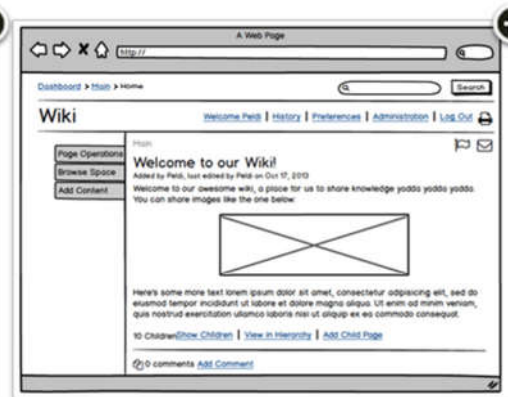
Web Sites



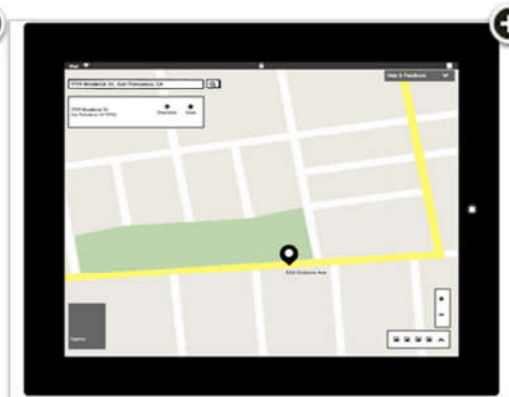
Dialog Windows



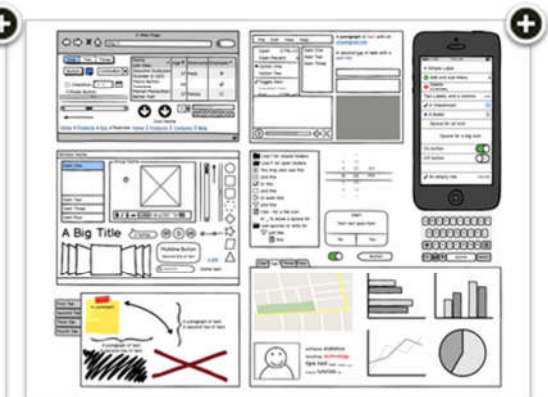
Desktop Apps



Web Apps



Tablet Apps



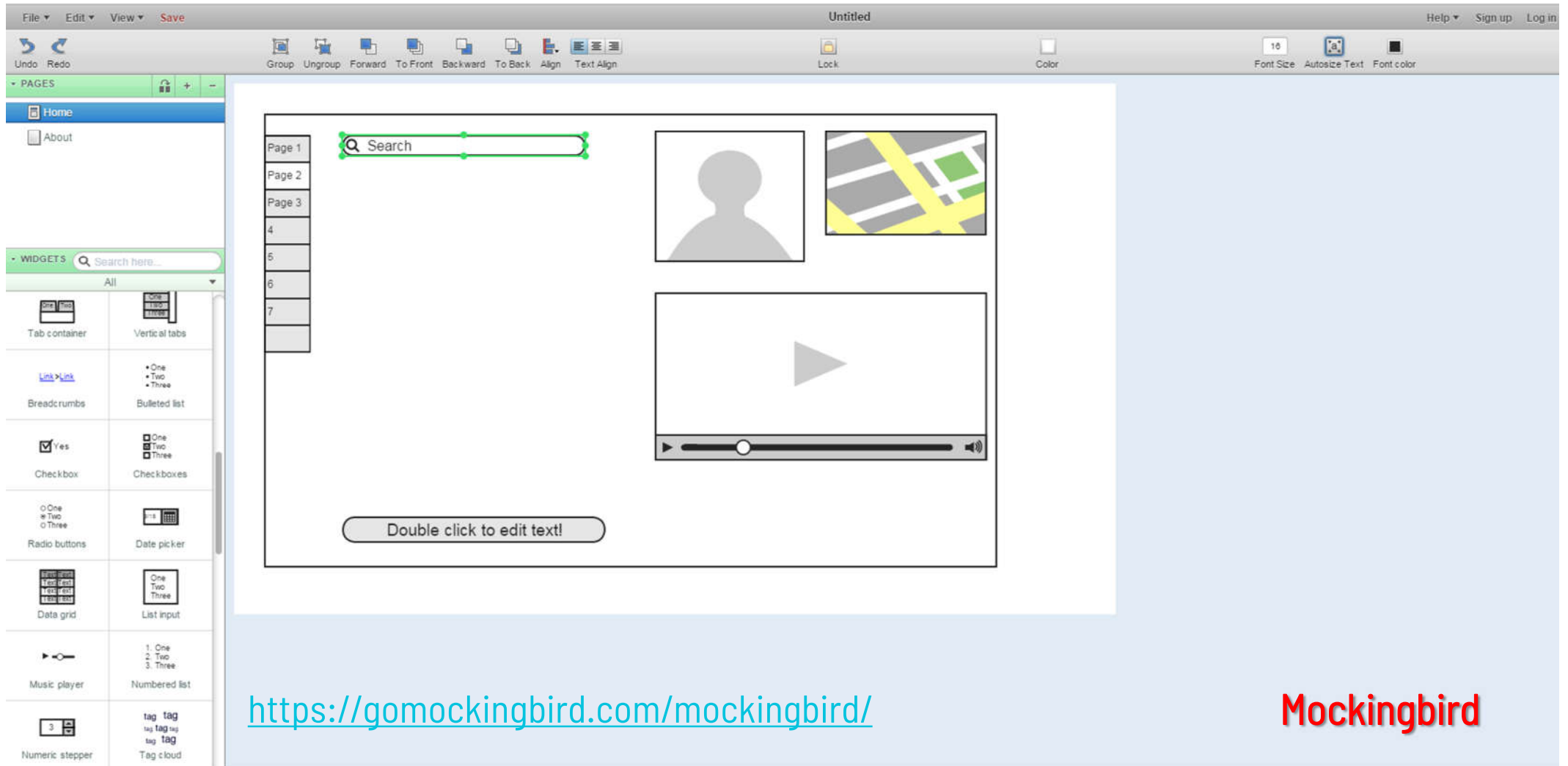
77 Built-In Controls

Balsamiq

<https://balsamiq.com/>

Kunden integrieren

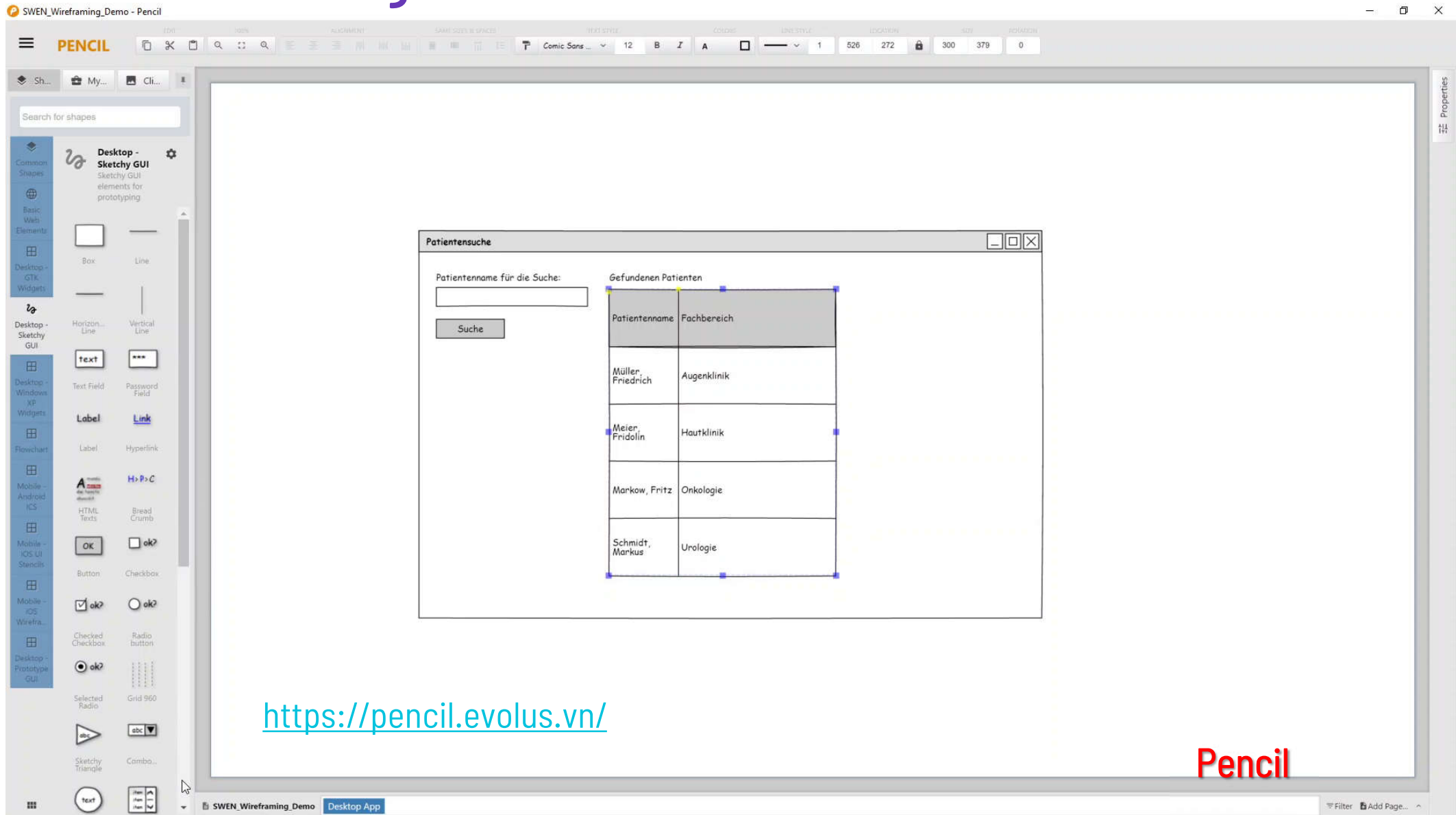
Wireframing: Frühes Feedback



Mockingbird

Kunden integrieren

Wireframing: Frühes Feedback



Kunden integrieren

Bewertung Wireframing

Für den Anwender: Software = Benutzeroberfläche.

- Praktisch – direkt mit Anwendern zusammen Software skizzieren, ohne Programmieren
- An Benutzeroberfläche treffen sich Realität und Software – Gemeinsamer Entwurf ermöglicht besseres Verständnis von Anwender und Entwickler füreinander

Aber Vorsicht:

- Auch Wireframing kostet Aufwand – Kosten/Nutzen-Verhältnis beachten
- Dem Softwareentwickler Freiheit lassen – nicht alle Wünsche sind pixelgenau umsetzbar
- Echte Geschäftslogik ist so kaum beschreibbar



ANFORDERUNGEN UND ANFORDERUNGS- MANAGEMENT:

MODELLIERUNG UND
DOKUMENTATION

Anforderungen und Anforderungsmanagement

Modellierung und Dokumentation

- Damit sie ihre Aufgabe erfüllen können, müssen Anforderungen **dokumentiert** werden
- Alle Stakeholder müssen die verwendeten Dokumentations-Technik akzeptieren und verstehen!
- Eine Auswahl üblicher und typischer Techniken, oft in Kombination verwendet:
 - **Use-Cases** (Anwendungsfälle) mit Diagrammen und Beschreibung in Formularen
 - In **natürlicher Sprache** beschriebene Anforderungen – strukturiert-schablonenbasiert
 - **Aktivitäts-, Sequenz- und Zustandsdiagramme** in UML

Modellierung und Dokumentation

Use Cases („Anwendungsfälle“)

- **Basis:** Beendete Erfassung des umzusetzenden fachlichen Zustands in Form einer Geschäftsprozessanalyse (also etwa Prozesse identifiziert und in Ablaufdiagramm visualisiert, Geschäftsregeln dokumentiert)
- Frage nun: Welche (Teil-)Prozesse werden zukünftig von welchem System unterstützt?

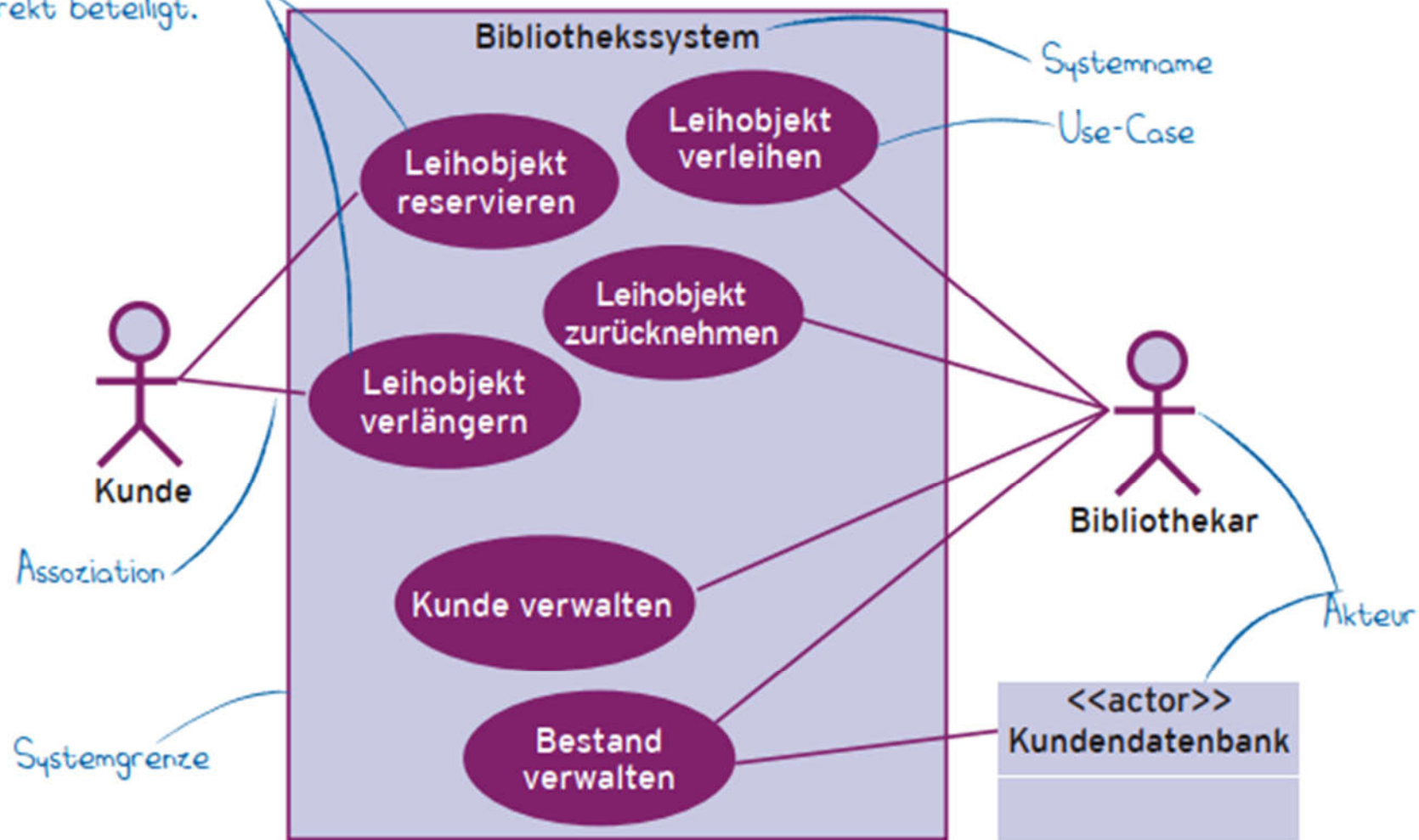
Ein **System-Use-Case** beschreibt eine typische Interaktion eines Anwenders mit einem System, die zu einem vom Anwender gewünschten Ergebnis führt. Er wird in einem System-Use-Case-Diagramm dargestellt, das meist durch eine textuelle System-Use-Case-Beschreibung ergänzt wird.

- System-Use-Cases werden durch **verfeinernde Use-Cases** detailliert.
- Use Cases werden in **Diagrammen** und **Spezifikationen** eingesetzt.

Modellierung und Dokumentation System-Use-Cases

Akteure: Anwender und
Nachbarn des Systems

An diesen beiden Use-Cases ist der Kunde
direkt beteiligt.

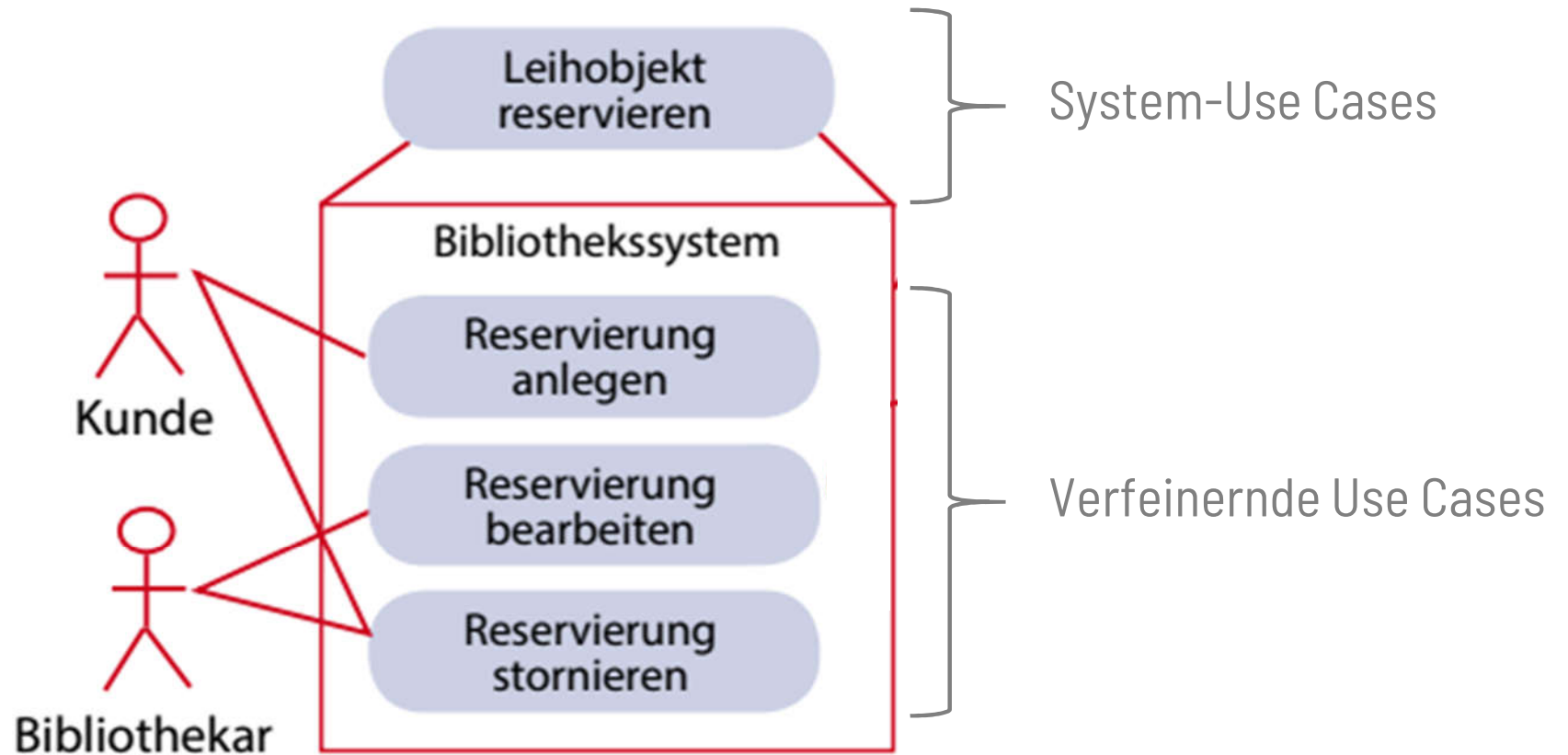


System-Use-Cases: Grobe Hauptfunktionen

Quelle: Rupp / SOPHISTen

Modellierung und Dokumentation

Verfeinernde Use Cases („Anwendungsfälle“)



Einsatz von verfeinernden Use Cases

Quelle: Rupp / SOPHISTen

Modellierung und Dokumentation

Spezifikation von Use Cases

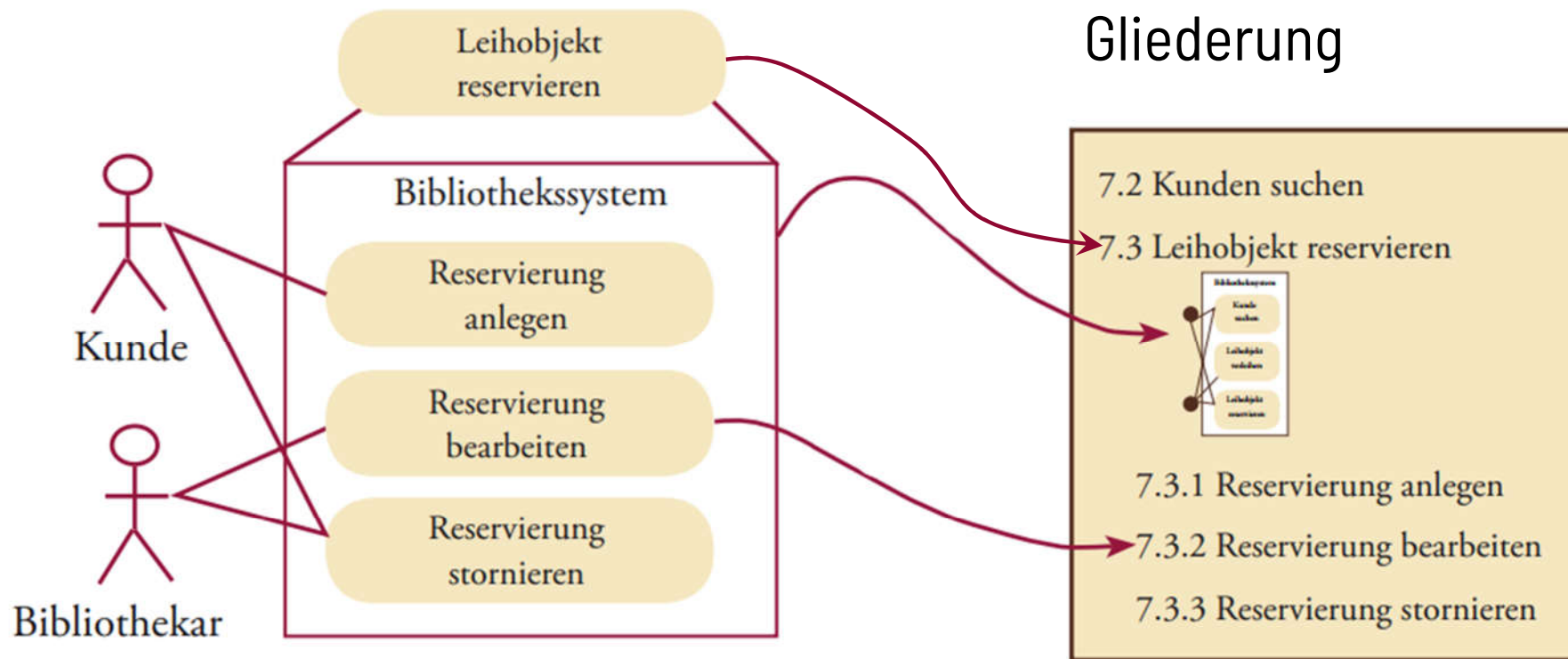
Use Case ID:			
Use Case Name:			
Created By:		Last Updated By:	
Date Created:		Date Last Updated:	

Actors:	
Description:	
Trigger:	
Preconditions:	
Postconditions:	
Normal Flow:	
Alternative Flows:	
Exceptions:	
Includes:	
Priority:	
Frequency of Use:	
Business Rules:	
Special Requirements:	
Assumptions:	
Notes and Issues:	

Modellierung und Dokumentation

Use Cases in Dokumenten

Einbettung von Use Cases in ein Dokument und Ableitung einer Gliederung

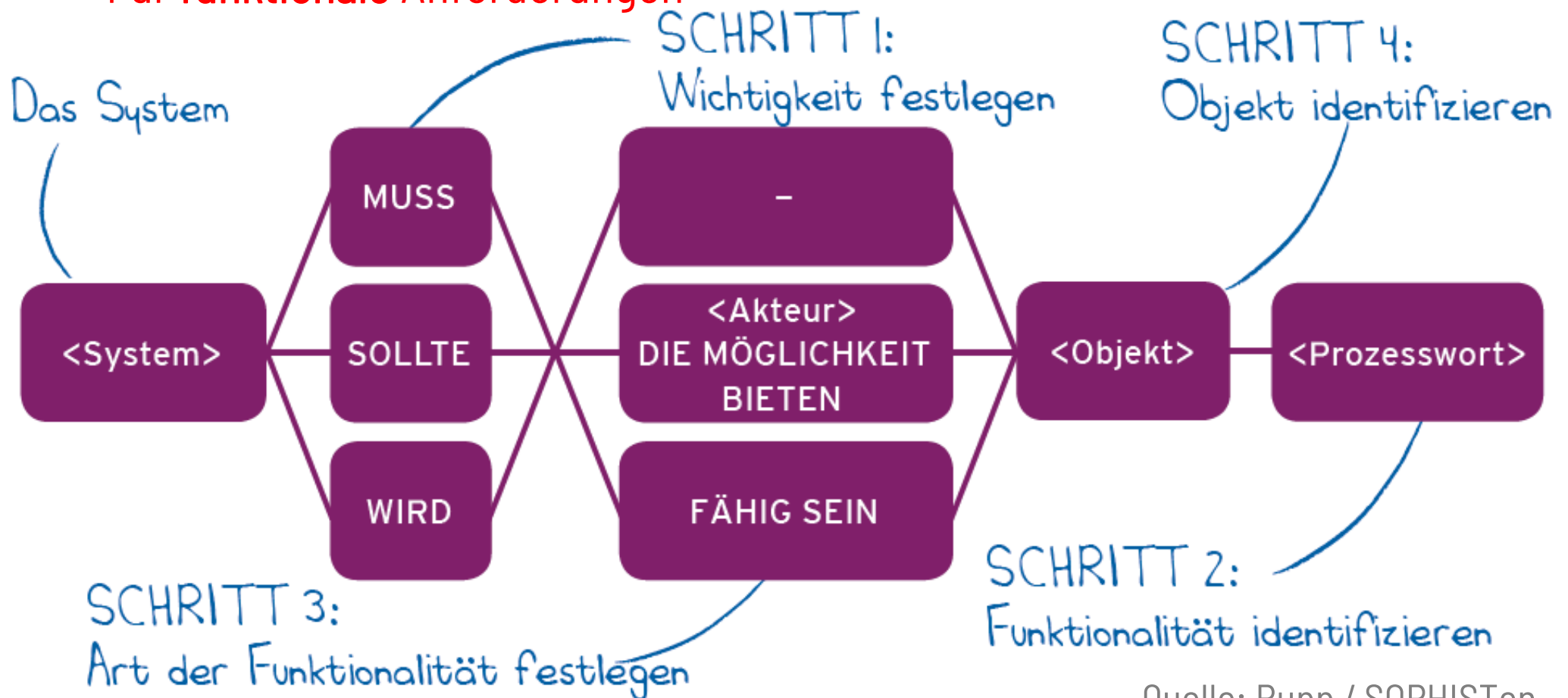


Quelle: Rupp / SOPHISTen

Modellierung und Dokumentation

Satzschablonen für natürliche Sprache

Für funktionale Anforderungen



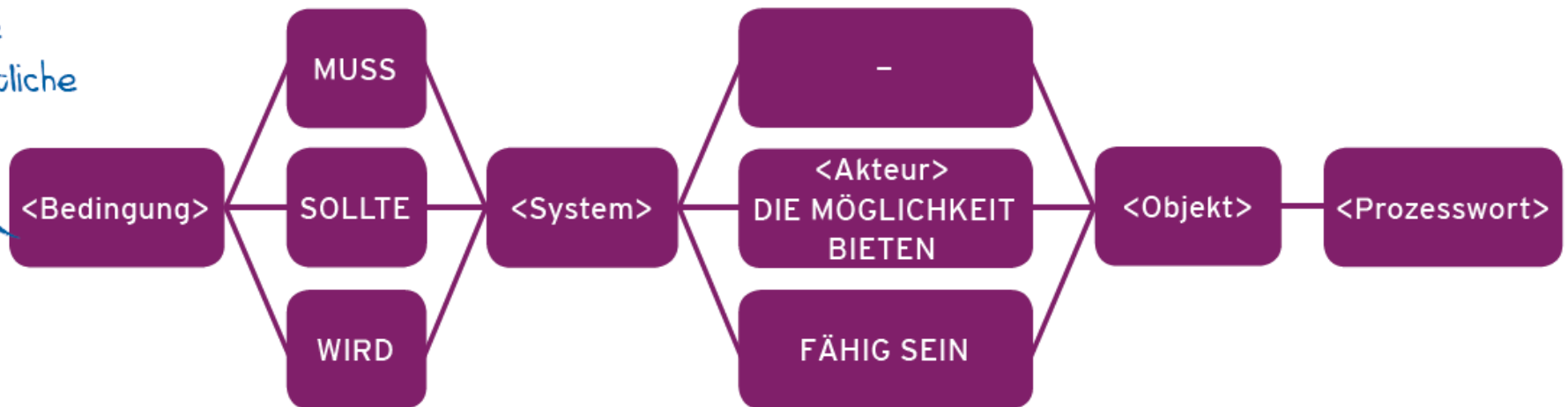
Quelle: Rupp / SOPHISTen

Modellierung und Dokumentation

Satzschablonen für natürliche Sprache, funktional

Für funktionale Anforderungen,
ergänzt um logische oder zeitliche Voraussetzungen

SCHRITT 5:
Formulieren Sie
logische und zeitliche
Bedingungen.



Quelle: Rupp / SOPHISTen

Modellierung und Dokumentation

Satzschablonen, funktional, Beispiele (1/2)

Beispiele für **funktionale** Anforderungen, die mit der Satzschablone konstruiert wurden:

Ohne Bedingung:

- Das Bibliothekssystem muss dem Administrator die Möglichkeit bieten, Leihobjekte von externen Datenmedien in den Datenbestand des Bibliothekssystems zu importieren.

Beispiel Typ 1: Selbsttätige Systemaktivität

- Falls der eingegebene Kunde bereits im System vorhanden ist, muss das System eine Fehlermeldung ausgeben.

Quelle: Rupp / SOPHISTen

Modellierung und Dokumentation

Satzschablonen, funktional, Beispiele (2/2)

Beispiele für **funktionale** Anforderungen, die mit der Satzschablone konstruiert wurden:

Beispiel Typ 2: Benutzerinteraktion

- Falls ein Kunde noch nicht im Bibliothekssystem vorhanden ist, muss das Bibliothekssystem dem Bibliothekar die Möglichkeit bieten, Kundendaten einzugeben.
- Falls ein Bibliothekskunde keine Leihobjekte entliehen hat, muss das Bibliothekssystem dem Bibliothekar die Möglichkeit bieten, diesen Bibliothekskunden zu löschen.

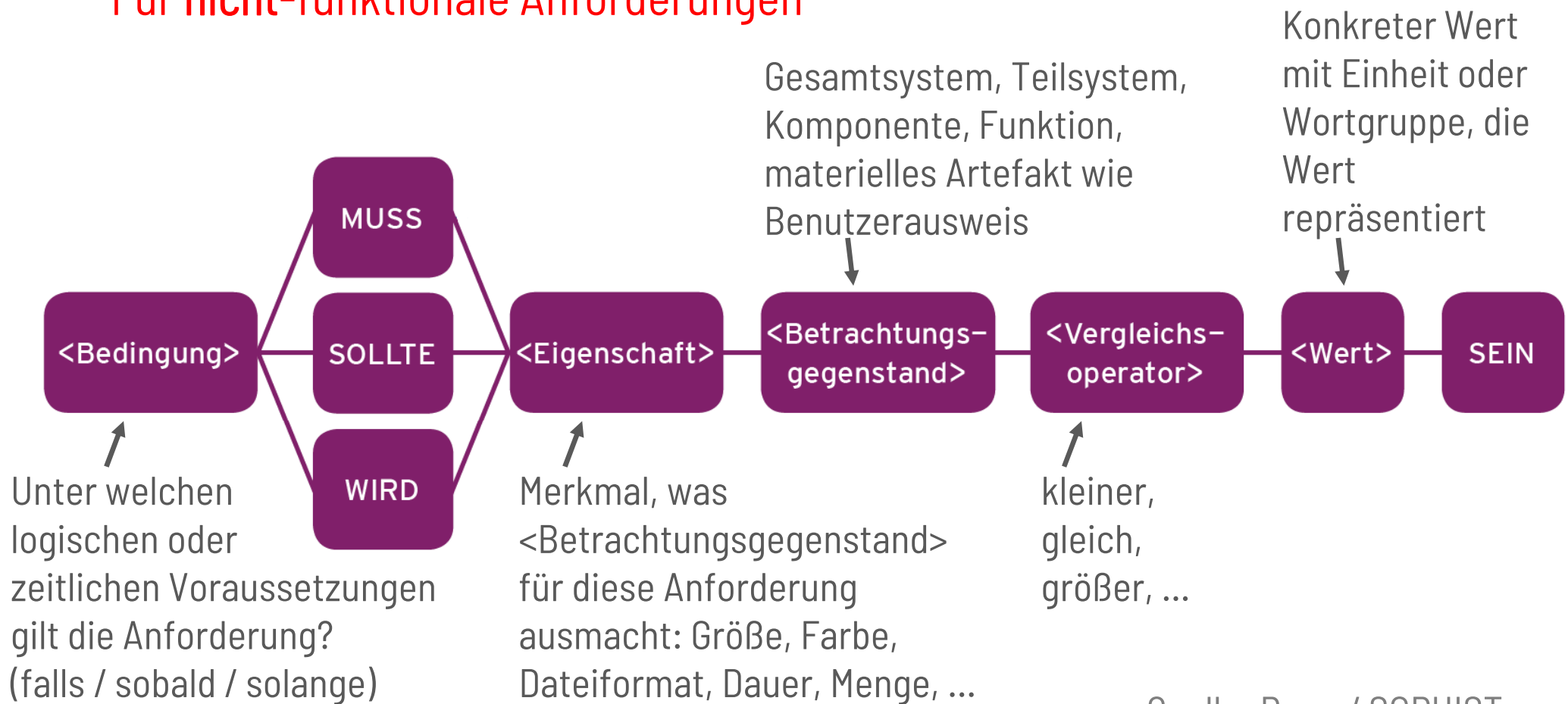
Beispiel Typ 3: Schnittstellenanforderung

- Solange das Bibliothekssystem in Betrieb ist, muss das Bibliothekssystem fähig sein, Daten für ein Software-Update zu empfangen. Quelle: Rupp / SOPHISTen

Modellierung und Dokumentation

Satzschablonen für natürliche Sprache, nicht-funktional

Für nicht-funktionale Anforderungen



Quelle: Rupp / SOPHISTen

Modellierung und Dokumentation

Satzschablonen, nicht-funktional, Beispiele

Beispiele für **nicht-funktionale** Anforderungen, die mit der Satzschablone konstruiert wurden:

Mit Bedingung, technische Anforderung:

- Während der Bürozeiten von 8-17 Uhr wird die Zahl gleichzeitiger Nutzer des Systems bis zu 100 sein.
- Während eines Jahres muss die Erreichbarkeit des Systems größer als 98% sein.

Beispiele ohne Bedingung (<Eigenschaft> rutscht nach vorne, Priorität muss/sollte/wird hinter <Betrachtungsgegenstand>):

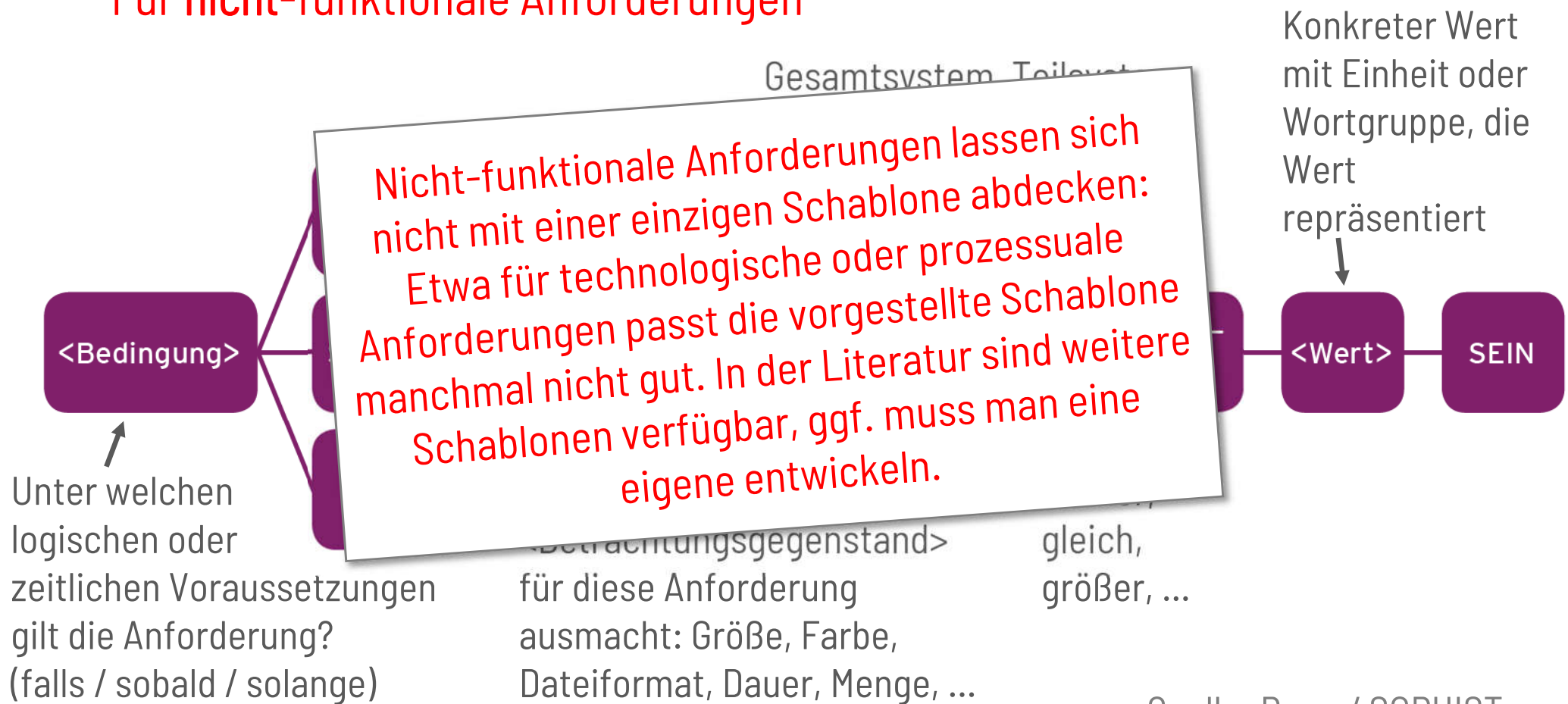
- Die Farbe des Benutzerausweises muss (gleich) weiß sein.
- Die Zeitdauer für die Übertragung auf einen anderen Server muss kleiner gleich 30 Minuten sein.

Quelle: Rupp / SOPHISTen

Modellierung und Dokumentation

Satzschablonen für natürliche Sprache, nicht-funktional

Für nicht-funktionale Anforderungen

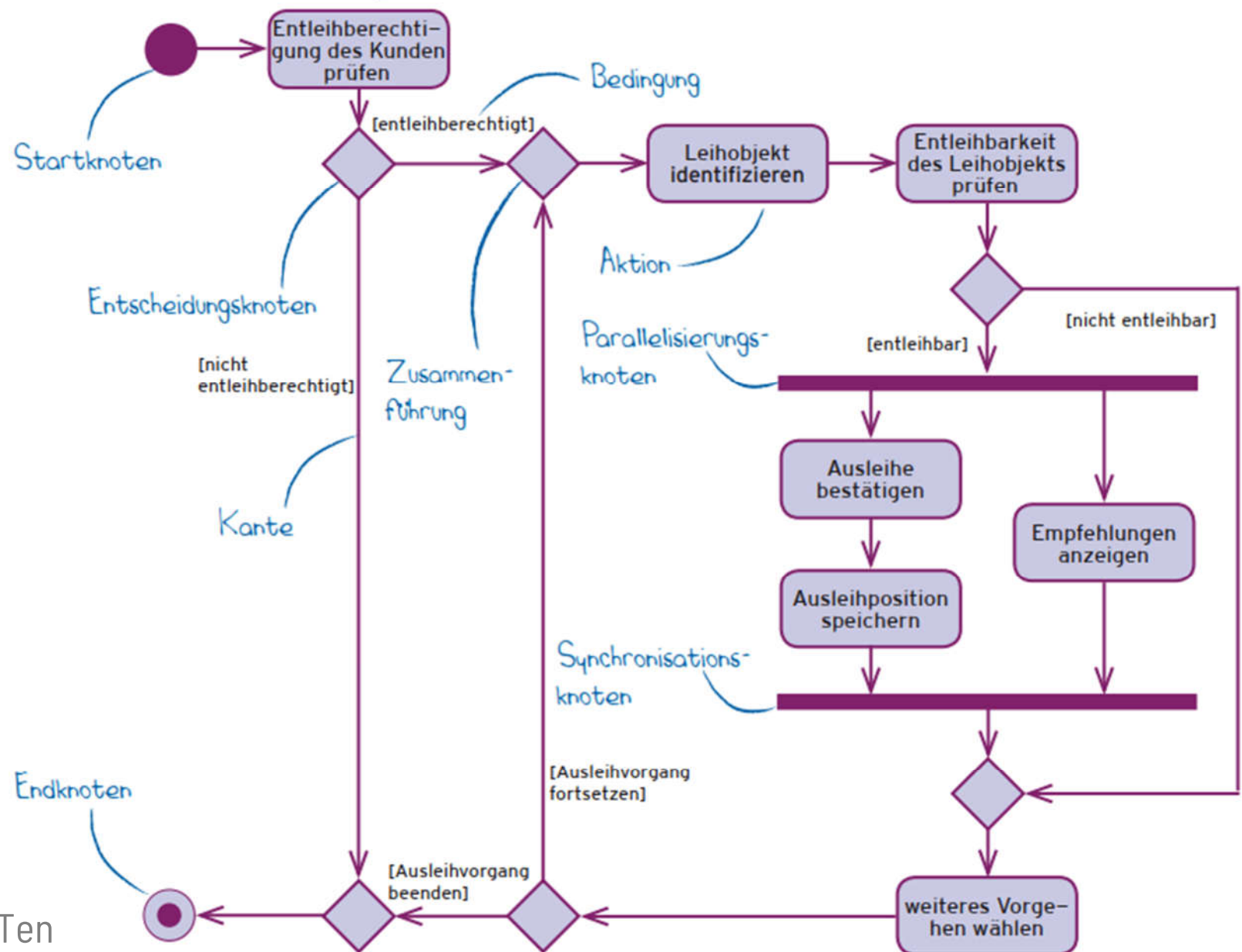


Quelle: Rupp / SOPHISTen

Modellierung und Dokumentation

Beispiel: Aktivitätsdiagramme in UML

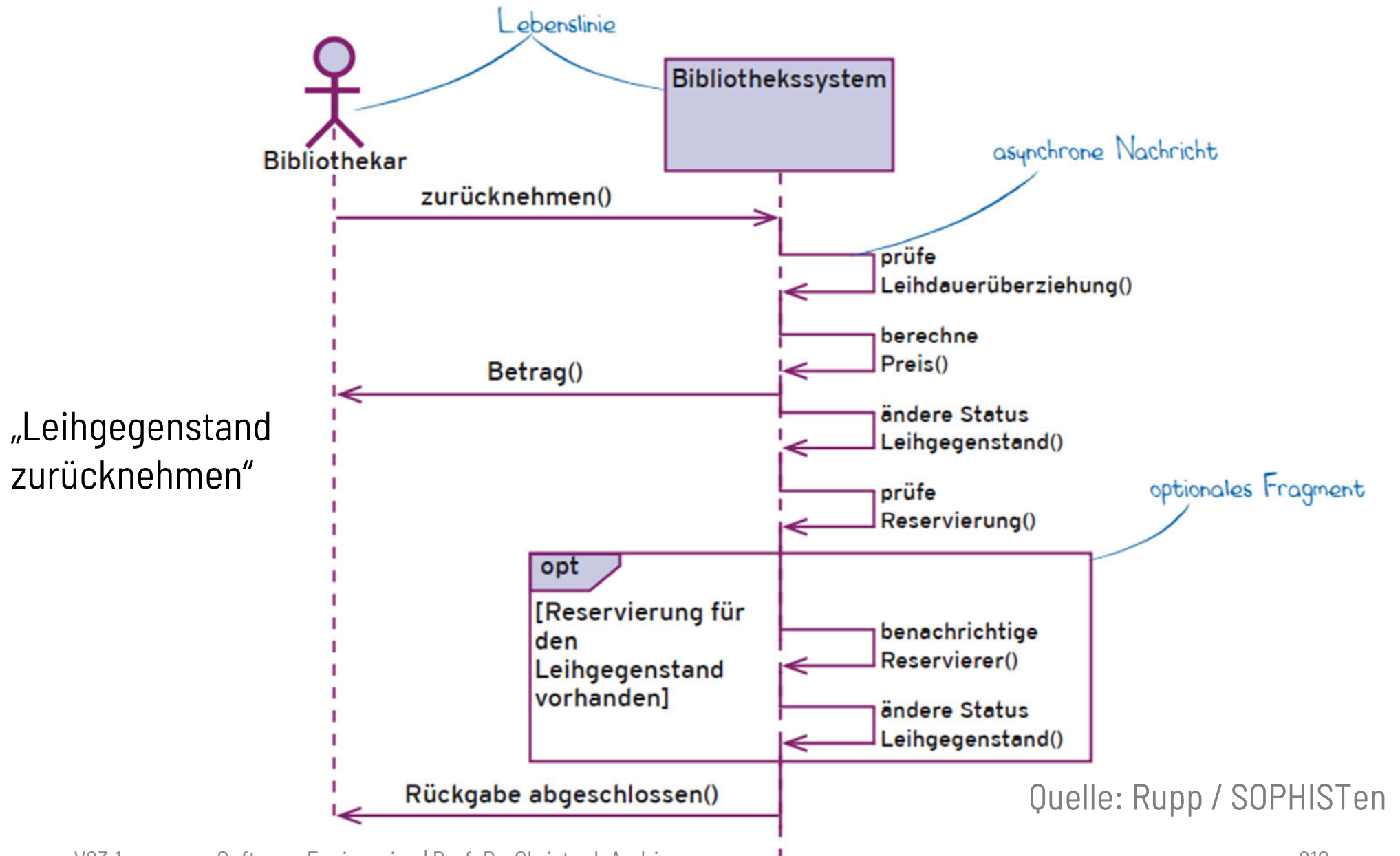
„Leihobjekt
verleihen“



Quelle: Rupp / SOPHISTen

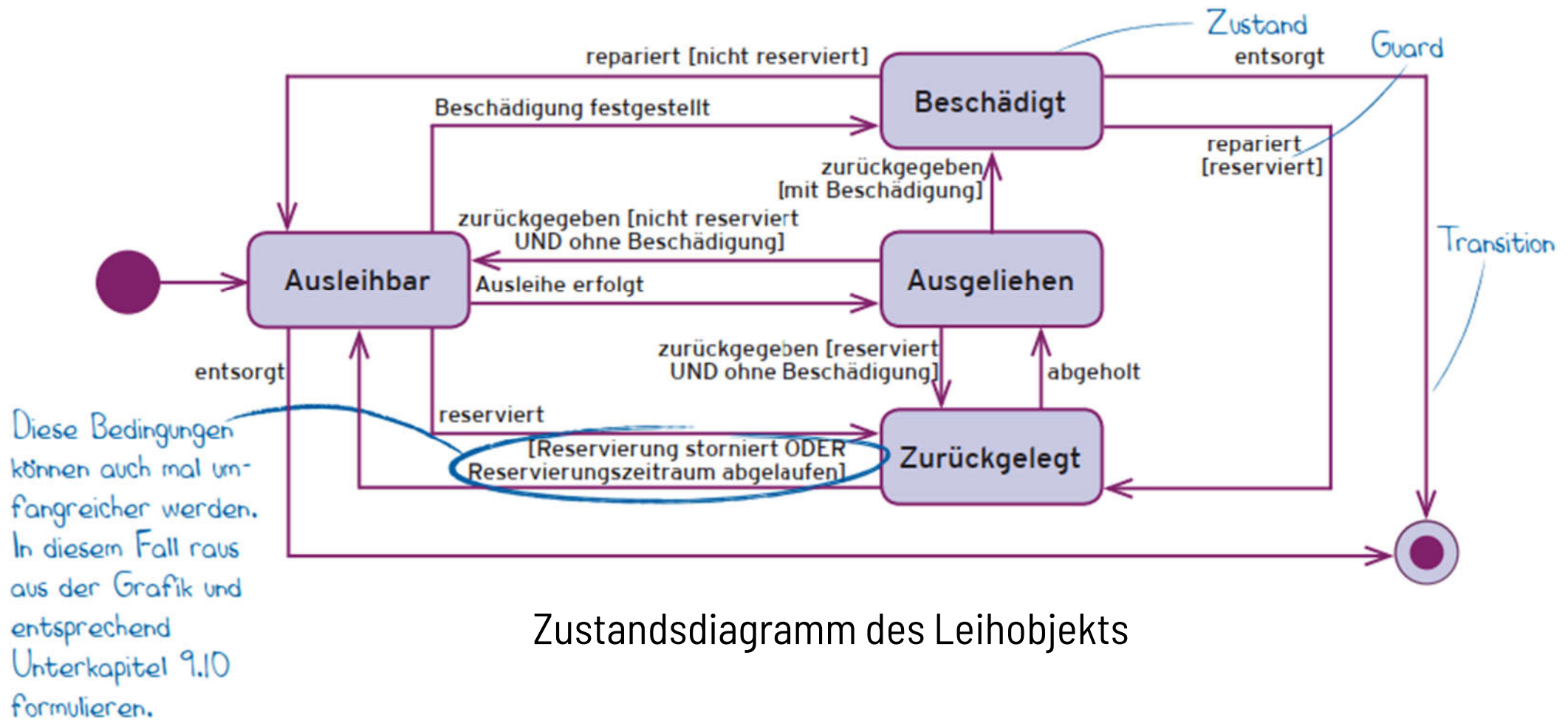
Modellierung und Dokumentation

Beispiel: Sequenzdiagramme in UML



Modellierung und Dokumentation

Beispiel: Zustandsdiagramme in UML



Zustandsdiagramm des Leihobjekts

Quelle: Rupp / SOPHISTen

┌ ANFORDERUNGEN UND ANFORDERUNGS- MANAGEMENT: ZUSAMMENFASSUNG

Wie geht man vor?

Methodik zusammengefaßt



- Qualitätskriterien für Anforderungen beachten
- Ziele festlegen (schon beim Projekt!), nicht Wege und Technik
- Die richtigen Anwender beteiligen: Stakeholder
- Frühes Feedback für Anwender mit Modellen der Benutzeroberfläche (Wireframing, Storyboarding)
- Mit System-Use-Cases beginnen, verfeinernde Use-Cases einsetzen
- Dokumentation formal-strukturiert: Tabellen, Sprachschablonen
- Diagramme ergänzen für Zustände, Aktivitäten, Sequenzen
- Tipps:
 - Branche verstehen und Prozesse hinterfragen, wo nötig
 - Es kann auch eine Verhandlung sein – immer den Wert vermitteln

Zusammenfassung

- Es gibt funktionale und nicht-funktionale Anforderungen
- Richtige Vorgehensweise bei Erhebung und Management von Anforderungen ist entscheidend
 - Analyse und Strukturierung
 - Sprachliche Präzision
 - Feedback (Wireframes, Storyboarding, Prototypen, Reviews)
- Qualitätskriterien und Best Practices helfen
- Achtung: Faktor Mensch!

